

UNIVERSIDAD DE COSTA RICA
SEDE CENTRAL, SAN PEDRO MONTES DE OCA

ESCUELA DE CIENCIAS DE LA COMPUTACIÓN E INFORMÁTICA

PROYECTO INTEGRADOR DE ARQUITECTURA Y ENSAMBLADOR
PROYECTO PROGRAMADO
GRUPO 4

ESTUDIANTES:

Marco Fernández Chacón - C32885

Adam Cortés Hernández - B92415

Jhonson Picado Vega - C15968

PROFESOR:

Willian Baltodano Cubillo

CIUDAD UNIVERSITARIA RODRIGO FACIO
II CICLO, 2025

Definición del problema

El riesgo de incendios no es una problemática de solo entornos industriales, sino que también afecta los entornos cotidianos. Estos, usualmente, se originan de flamas pequeñas que, al no ser controladas a tiempo, se pueden convertir en los peores incendios. Es cierto que muchas flamas comienzan siendo pequeñas o poco intensas, por lo que se consideran poco peligrosas, pero su amenaza aumenta si también lo hace su temperatura y podrían propagarse, lo que pone en riesgo la seguridad de personas, bienes materiales e instalaciones.

Esto puede ocurrir en entornos domésticos como cocinas, talleres o, en general, en lugares con velas, estufas o encendedores. De igual manera, podría ocurrir en zonas industriales, tales como plantas de producción, almacenes con materiales inflamables e, incluso, laboratorios o espacios con maquinaria que genera calor.

La necesidad se encuentra en poder detectar tempranamente la presencia de una flama, clasificar su nivel de riesgo y responder adecuadamente. Aunque ya existen sistemas convencionales, como alarmas o sensores de humo, que buscan facilitar la respuesta y disminuir el riesgo en caso de incendios, estos podrían no ser suficientes para detectar y clasificar llamas individuales y más pequeñas. Además, la respuesta a estas llamas dependen de la intervención humana, muchas veces calificada, por lo que estas situaciones pueden escalar rápidamente si no se cuenta con el conocimiento y equipo adecuados.

Propuesta de solución al problema

A raíz de la problemática señalada, se justifica la necesidad de implementar un sistema inteligente que integre sensores de flama y temperatura que sea capaz de diferenciar entre una flama de baja intensidad de una de alta peligrosidad. De esta manera, el sistema podría emitir advertencias preventivas cuando el riesgo es moderado y, en casos críticos, podría activar un mecanismo automático, una torreta de agua controlada por un servomotor,

para sofocar la flama. De este modo, se busca incrementar la seguridad en entornos domésticos e industriales, reducir los tiempos de respuesta y prevenir la propagación de incendios al ofrecer una herramienta práctica y eficiente para la protección de personas, bienes e instalaciones.

La propuesta de solución consiste, entonces, en el desarrollo de un sistema capaz de detectar y responder a incendios en etapas tempranas mediante sensores de flama y temperatura conectados a un microcontrolador Arduino UNO R3, el cual, gracias a sus capacidades de integración para sistemas, permite implementar este diseño fácilmente. Así, este procesa la información para clasificar el nivel de riesgo y, según la situación, emitirá una advertencia a través de una pantalla LCD o activará un servomotor con una torreta de agua para intentar apagar la flama. Con esto, se busca reducir la dependencia exclusiva de la intervención humana especializada y ofrecer una alternativa práctica, automatizada y de bajo costo para mejorar la seguridad en entornos domésticos e industriales.

Arduino permite la implementación de una solución así porque cuenta con un lenguaje de programación simple, accesible y con librerías diversas que permite reducir el tiempo de desarrollo. Asimismo, sus entradas analógicas y digitales y sus salidas digitales/PWM facilita la interacción con sensores y actuadores, lo que permite leer los sensores y controlar el servomotor y la pantalla LCD de manera óptima. Finalmente, Arduino tiene una gran capacidad de escalabilidad gracias a un hardware amigable, por lo que se pueden incluir funciones específicas al agregar módulos.

Para este proyecto, los componentes a usar serán:

- Una pantalla LCD para Arduino.
- Tres sensores de flama/NIR.
- Un sensor de temperatura
- Un servomotor

- Un buzzer de sonido

Diagrama de entradas y salidas

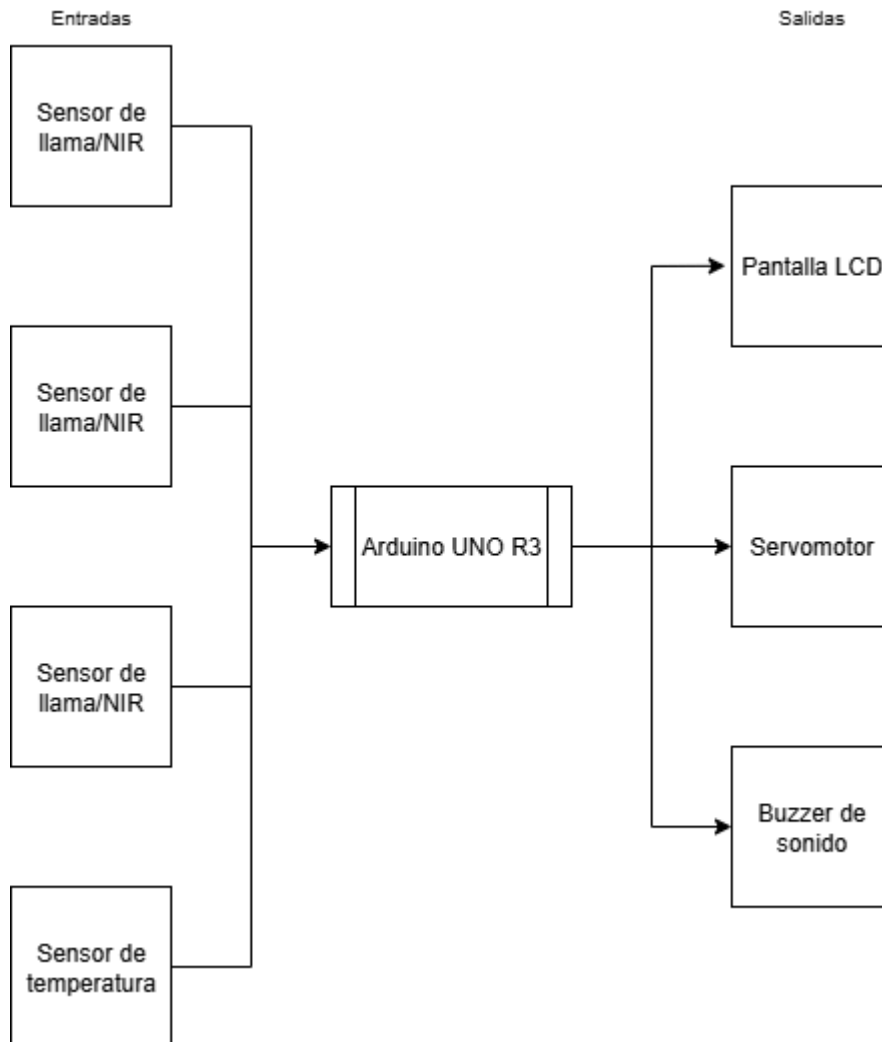
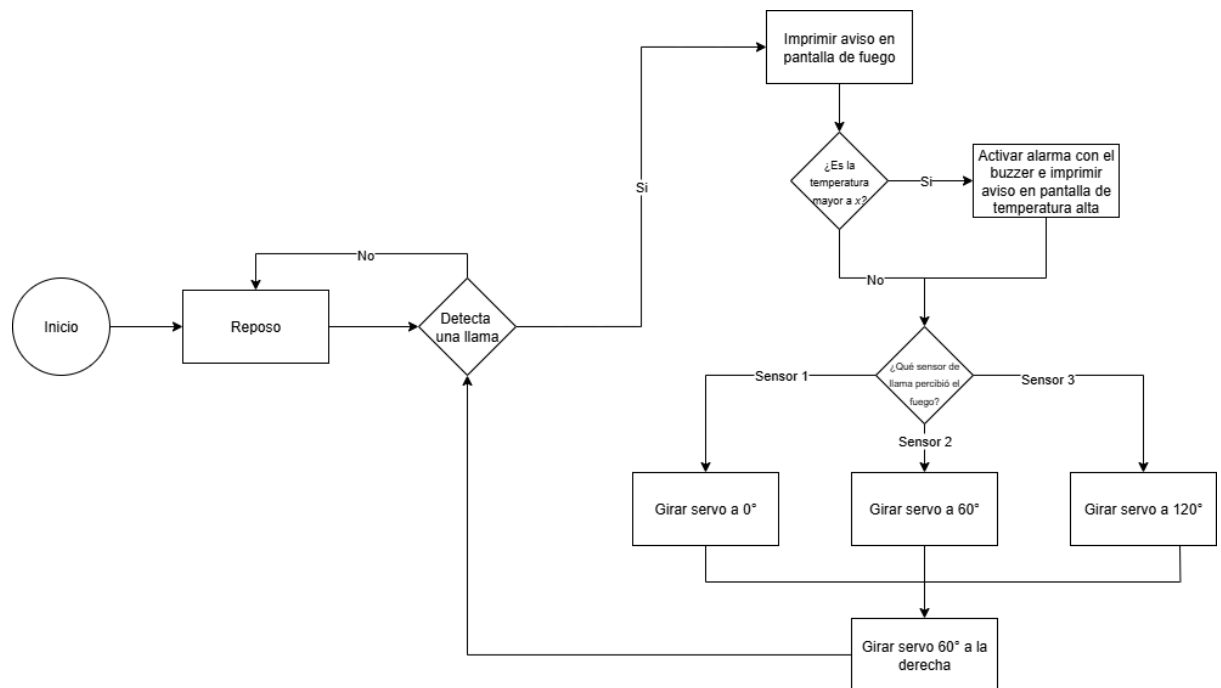


Diagrama de estados



Introducción

Este proyecto busca integrar un sistema inteligente, definido como una máquina tecnológica avanzada, el cual percibe el mundo que lo rodea por medio de sensores y otros componentes, de forma que genere una respuesta a través de actuadores que forman una salida (University of Nevada, Reno, s/f). El problema a resolver será integrado por medio de la utilización de un microcontrolador Arduino UNO R3, el cual ha sido diseñado como introducción a la electrónica y la programación, que presenta un procesador principal ATmega328P a 16 MHz y una especificación de memoria de 2KB de SRAM, 32KB de memoria flash y 1KB de EEPROM. Además, presenta un procesador ATmega16U2 a 16 MHz que controla la comunicación USB-Serial (*UNO R3*, s/f).

De igual manera, este microcontrolador tiene 14 pines de entrada y salida digital, 6 pines de entrada analógica con una resolución de 10 bits en un rango de 0 a 1023 y 6 pines que admiten PWM (*Pulse Width Modulation*) para salida analógica (Arduino, s/f). Estos últimos pines serán usados para la comunicación con componentes digitales y solo presentan

dos estados (alto o bajo), significando la presencia o ausencia de corriente en el componente. A su vez, los componentes analógicos emiten una señal eléctrica variable a lo largo del tiempo (Soneira, 2016). Estos componentes reciben energía por medio de los pines de voltaje que se encuentran en la placa microcontroladora, los cuales tienen una salida de 5V o 3.3V (Arduino, s/f).

La placa Arduino UNO R3 hace uso de la Arduino API, una versión simplificada de C y C++ usada para programar y cargar las instrucciones a la placa por medio de su puerto USB. Un algoritmo funcional para esta placa necesita, como requerimiento, el uso de las funciones *void setup()* y *void loop()*: la primera está encargada de correr una única vez al encender la placa y define el estado de cada pin y si este será usado como entrada o salida, la velocidad de la comunicación serial y las inicializaciones de bibliotecas; en cambio, la segunda ejecuta el bucle de código donde se encuentra la solución al problema planteado, la cual se ejecutará una y otra vez (Söderby, s/f).

Proceso de diseño para la máquina de estados

El sistema fue diseñado utilizando una máquina de estados finitos (FSM) para organizar las acciones dependiendo de las condiciones detectadas por los sensores de flama y temperatura. Los estados principales definidos fueron los siguientes:

1. **SEARCHING_FLAMES** (búsqueda de llamas):

Estado inicial donde el sistema monitorea constantemente los sensores de flama.

2. **CHECKING_TEMPERATURE** (revisión de temperatura):

Si se detecta una llama, se mide su temperatura. Si es demasiado alta, se activa una alarma con el buzzer y se muestra un aviso en pantalla.

3. **EXTINGUISHING_FLAMES** (extinción de llamas):

El sistema mueve el servomotor hacia la posición correspondiente (0° , 60° o 120°), dependiendo de si el sensor detecta una llama, y simula el proceso de extinción.

Del mismo modo, el flujo de estados del sistema comienza en `SEARCHING_FLAMES`:

1. Si no detecta llama, se mantiene en el mismo estado.
2. Si detecta llama, pasa a `CHECKING_TEMPERATURE`.
3. Luego pasa a `EXTINGUISHING_FLAMES` y, al finalizar, retorna a `SEARCHING_FLAMES`.

Implementación de la lógica en Arduino IDE

La máquina de estados se implementó en Arduino IDE mediante un switch-case, utilizando un enum para definir los estados posibles. Así, la estructura del código es la siguiente:

`setup()`: inicializa la comunicación serial.

`loop()`: contiene el switch-case que ejecuta las acciones de cada estado.

`isThereAFlame()`: revisa los sensores de flama.

`flameAlarm()`: evalúa la temperatura y activa la alarma si es necesario.

`extinguishFlames()`: simula el movimiento del servo y la extinción de la llama.

`simulation()`: genera valores aleatorios para simular las entradas de los sensores.

Simulación del diseño en alto nivel de la aplicación en el Arduino IDE

Debido a que aún no se utilizan los componentes de hardware necesarios para el funcionamiento de la torreta, se definieron variables que representan los diferentes sensores del sistema, entre ellos tres sensores de flama con un rango de detección de 60° cada uno y un

sensor de temperatura ambiental. En lugar de realizar lecturas directas desde pines físicos mediante instrucciones como `digitalRead()` o `analogRead()`, las variables fueron alimentadas con valores generados por la función `random()`. De esta forma, se simulan escenarios de detección de flamas y variaciones de temperatura en cada ciclo de ejecución del programa.

Asimismo, para sustituir las salidas físicas, se utilizó el monitor serial del Arduino IDE. Mediante la instrucción `Serial.println()` se imprimen mensajes que describen las acciones que realizaría el sistema, tales como la detección de una llama, la activación de una alarma o el movimiento del servo encargado de extinguir el fuego. Esto brinda una visualización clara del flujo de ejecución y permite validar que el sistema responde adecuadamente a las condiciones simuladas.

Finalmente, se implementó una función denominada `simulation()`, la cual se encarga de generar entradas de prueba en cada iteración del bucle principal. Con esta función, el sistema se enfrenta a diferentes combinaciones de detecciones de flama y niveles de temperatura, emulando el comportamiento dinámico del entorno real. De esta manera, se logra probar la robustez de la lógica de control sin requerir aún la conexión de sensores o actuadores físicos.

Modelo y Especificaciones Técnicas

Para el proyecto utilizamos los siguientes componentes:

- SM-S2309S MOTOR
 - **Especificaciones Técnicas:**
 - Modulación: Análogo
 - Voltaje de operación: 4.8V – 6V
 - Velocidad: 0.12 s/60° a 4.8V
 - Tipo de señal: PWM

- Rango de giro: $0^{\circ} - 180^{\circ}$
 - **Funcionalidad relevantes para la aplicación**
 - Permite dirigir el servomotor a la llama detectada para empezar el proceso de extinción de la misma.
 - **Cómo se integran en la programación en Arduino**
 - Se programa usando la librería Servo.h
 - Señal: pin digital
 - VCC: 5V
 - GND: GND
- Sensor de flama KY-026
 - **Especificaciones Técnicas:**
 - Voltaje de operación: $3.3V - 5V$
 - Salida: Digital (0/1) y analógica
 - Detección: luz visible o infrarroja de llama
 - Distancia de detección: $\sim 0-80$ cm
 - **Funcionalidad relevante para la aplicación**
 - Detecta la presencia de llama o fuego.
 - **Cómo se integran en la programación en Arduino**
 - VCC $\rightarrow$ 5V
 - GND $\rightarrow$ GND
 - DO (digital) $\rightarrow$ pin digital
 - AO (analógico) $\rightarrow$ pin analógico

- Sensor de temperatura TMP36
 - **Especificaciones Técnicas:**
 - Voltaje de operación: 2.7V – 5.5V
 - Rango de temperatura: -40°C a +125°C
 - Salida: analógica, 10 mV/°C
 - Precisión: $\pm 2^{\circ}\text{C}$
 - **Funcionalidad relevante para la aplicación**
 - Detecta la temperatura actual de la llama para evaluar si nivel de amenaza
 - **Cómo se integran en la programación en Arduino**
 - VCC $\rightarrow$ 5V
 - GND $\rightarrow$ GND
 - Vout $\rightarrow$ pin analógico

- Pantalla LCD LCM1602C
 - **Especificaciones Técnicas:**
 - Voltaje de operación: 5V
 - Dimensiones: 16x2 caracteres
 - Interfaz: paralelo (8 bits)
 - Retroiluminación LED: incluida
 - **Funcionalidad relevante para la aplicación**
 - Permite mostrar información en tiempo real sobre las acciones del sistema.
 - **Cómo se integran en la programación en Arduino**
 - Conexión típica (modo 4 bits):

- RS → pin digital
- E → pin digital
- D4-D7 → pines digitales
- VCC → 5V
- GND → GND

- Buzzer PKM13EPYH4000-A0

- **Especificaciones Técnicas:**

- Voltaje de operación: 3 – 20 VDC
- Frecuencia resonante: 4000 Hz
- Corriente máxima: 5 mA
- Nivel de sonido: ~85 dB a 10 cm / 4 kHz
- Dimensiones: 13 mm de diámetro
- Tipo de montaje: THT

- **Funcionalidad relevante para la aplicación**

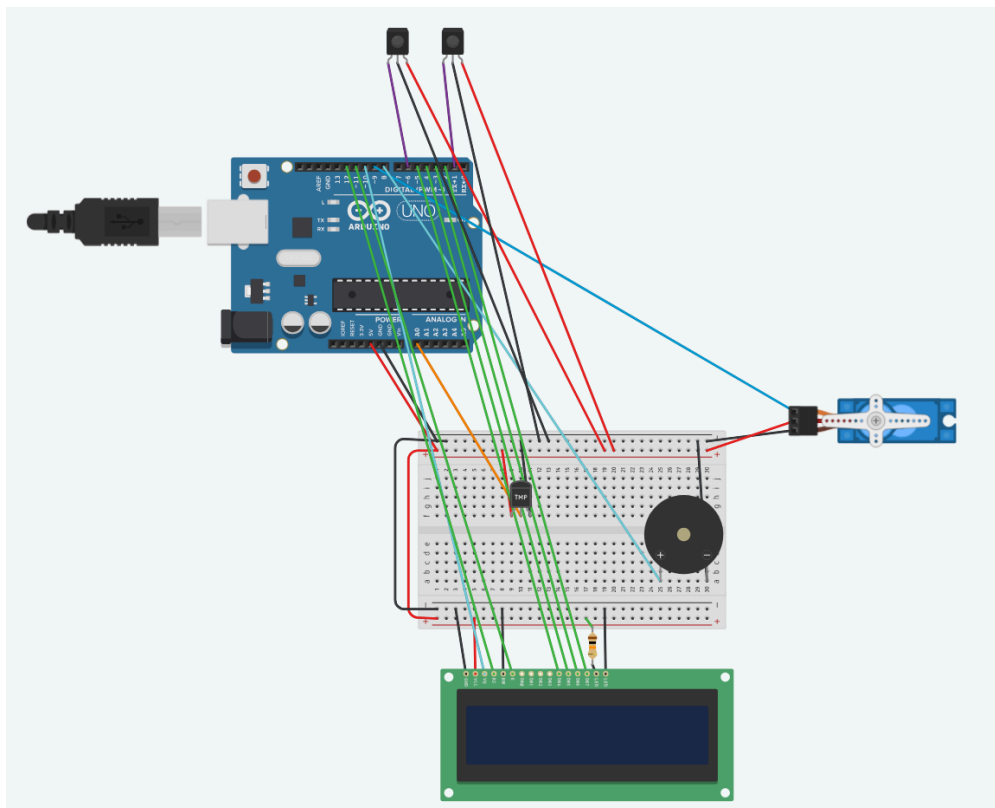
- El buzzer emite un sonido de alerta cuando el sistema detecta una llama o una temperatura anormalmente alta.

- **Cómo se integran en la programación en Arduino**

- Se puede controlar utilizando la función tone() y noTone() del entorno Arduino.
- Pin de señal → Pin digital
- VCC → 5V
- GND → GND

Esquemático de la Versión Funcional

En la figura se presenta el diagrama de conexión del sistema implementado en Tinkercad, utilizando la placa Arduino UNO R3 como controlador principal. En esta etapa, se integraron los sensores y actuadores principales del sistema de detección y respuesta ante fuego, conformado por el sensor de flama KY-026, el sensor de temperatura TMP36, el servomotor SM-S2309S, el buzzer PKM13EPYH4000-A0 y la pantalla LCD LCM1602C.



Estructura general del código desarrollado en Arduino

Inicialización (setup())

- Se importan las librerías Servo.h y LiquidCrystal.h.
- Se configuran los pines correspondientes a sensores, servomotor, buzzer y pantalla LCD.
- El servomotor se coloca en posición inicial (0°).
- La pantalla LCD muestra el mensaje “Sistema iniciado” al encenderse.

Ciclo principal (loop())

El sistema ejecuta un bucle infinito que depende del estado actual de la máquina:

- **SEARCHING_FLAMES**
 - Se monitorean los tres sensores de flama mediante `digitalRead()`.
 - Si alguno detecta una llama, el sistema cambia al estado **CHECKING_TEMPERATURE**.
 - Si no se detectan anomalías, la pantalla muestra el mensaje:
“No se han detectado llamas”.
- **CHECKING_TEMPERATURE**
 - Se lee el valor del sensor TMP36 usando `analogRead(A0)` y se convierte a °C.
 - Si la temperatura supera el umbral definido (`baselineTemp = 24.0°C`), se activa el buzzer mediante `tone(8, frecuencia)`.
 - La pantalla LCD muestra “ALERTA llama alta temperatura”.
 - Luego, el sistema pasa al estado **EXTINGUISHING_FLAMES**.
- **EXTINGUISHING_FLAMES**
 - El servomotor se posiciona en el ángulo correspondiente a la fuente detectada (0°, 90°, 120°, etc.) usando `extinguishServo.write(ángulo)`.
 - Se simula la extinción y se reinicia la búsqueda.
 - La pantalla LCD indica “Extinguiendo llama...”.

Funciones Adicionales

- `isThereAFlame()`: Detecta si alguno de los sensores de flama está activo.
- `flameAlarm()`: Evalúa la temperatura y activa el buzzer si excede del límite seguro.
- `extinguishFlames()`: Mueve el servomotor para apuntar hacia donde este el fuego detectado
- `rotateServo(int angle)`: Controla el desplazamiento del servo.

Conexión de la placa Arduino UNO R3 con el hardware

Componente	Tipo de señal	Pin Arduino	Descripción / Función
Sensor de flama 1 (0°–60°)	Digital	D6	Detecta presencia de llama en el primer sector.
Sensor de flama 2 (60°–120°)	Digital	D1	Detecta presencia de llama en el segundo sector.
Sensor de flama 3 (120°–180°)	Digital	D0 (<i>definido, pero no utilizado en esta etapa</i>)	Detecta presencia de llama en el tercer sector.
Sensor de temperatura TMP36	Analógica	A0	Mide la temperatura de la fuente de calor.
Servomotor SM-S2309S	PWM	D9	Se orienta hacia la posición del fuego detectado.
Buzzer PKM13EPYH4000-A0	Digital (PWM)	D8	Genera señal sonora de alerta.
Pantalla LCD LCM1602C	Digital	RS = D12, E = D11, D4 = D5, D5 = D4, D6 = D3, D7 = D2	Muestra mensajes e información del sistema.
Alimentación general	–	5V y GND	Distribuye voltaje y referencia de tierra a todo el circuito.

Interconexión general del sistema:

- Sensor de temperatura conectado al pin A0 del Arduino, mientras que los pines laterales se conectan a 5V y GND.
- Dos sensores de flama están conectados a los pines D1 y D6 del Arduino. Cada sensor tiene su pin de salida digital, alimentado con 5V y GND. Detectan la presencia de fuego y envían una señal alta o baja según corresponda.
- Pantalla LCD conectada a los pines D2, D3, D4, D5, D11, D12, 5V y GND. Muestra la información de temperatura, estado del sistema y detección de flama.

- Buzzer conectado al pin D8 del Arduino. Recibe una señal PWM que genera el tono de alerta cuando se detecta fuego o temperatura anormal.
- Servomotor conectado al pin D9, con alimentación de 5V y GND. Su función es simular el movimiento de un sistema de extintor.
- Resistencia de 220 Ω utilizada para limitar la corriente del LCD, específicamente en la línea del pin de contraste (V0).

Optimización de la aplicación mediante Lenguaje Ensamblador

Selección del módulo

El módulo seleccionado para la optimización corresponde a la lectura del sensor de temperatura y a la detección de intensidad de llama, lo cual está implementado en la función ‘flameAlarm()’. Este código constantemente revisa la temperatura creada por la llama para decidir si debe enviar una alarma si su temperatura es mayor a cierta cantidad.

Este módulo fue elegido debido a que se ejecuta con bastante frecuencia cuando se detecta una llama, es muy importante para que el programa pueda tomar decisiones y además involucra operaciones de conversión analógica y cálculo matemático, donde el tiempo de muestreo y procesamiento es relevante.

Análisis del comportamiento actual del módulo

En la versión original, la función ‘flameAlarm()’ realizaba la lectura del sensor térmico mediante ‘analogRead()’, una función de Arduino para la conversión analógica-digital.

Este método es simple y fácil de usar, pero introduce una latencia considerable debido a las rutinas de configuración y espera que maneja internamente.

- `analogRead()` tarda aproximadamente 110 μ s.
- Con la frecuencia de ejecución del loop, aumenta el tiempo de respuesta del sistema ante una llama intensa.
- La lectura es precisa, pero al costo de usar más tiempo, lo cual afecta la eficiencia.

Propuesta y justificación de optimización

Sustituir el uso de `analogRead()` por una configuración y control directo del ADC mediante registros AVR (`ADMUX`, `ADCSRA`, `ADCL`, `ADCH`). De esta manera, se evita la sobrecarga de funciones intermedias y se controla el proceso de conversión directamente a nivel de hardware.

Se decidió optimizar el módulo de lectura del sensor de temperatura, ya que será una de las partes más críticas del sistema al ejecutarse constantemente y afectar directamente el tiempo de respuesta ante una posible llama. En su forma original, el uso de `analogRead()` implicará una sobrecarga innecesaria por las rutinas internas de Arduino, reduciendo la eficiencia del proceso. Al reescribir la lectura en ensamblador AVR y manipular directamente los registros del ADC, se obtendrá un control más preciso y una reducción notable en el tiempo de conversión, mejorando la velocidad y el rendimiento general del sistema.

De esta forma, se espera obtener los siguientes resultados:

Aspecto	Antes (Sin Ensamblador)	Después (Con Ensamblador)
Tiempo de lectura ADC	~110 μ s	~30 μ s
Precisión	Alta	Alta
Sobrecarga de funciones	Alta	Baja
Eficiencia	Media	Alta

Explicación del código en ensamblador

La implementación de las optimizaciones se efectuó en dos bloques de código, durante el proceso de inicialización del Arduino, el método *setup()*, se configuró el registro ADMUX con su bit de REFS0 en 1, para indicar que el proceso de conversión usará 5V como referencia para la división del voltaje de entrada, esto debido a que el sensor de temperatura TMP36 trabaja en voltajes 0 a 5V. La configuración del registro ADMUX de esta manera también se asegura que los 4 bits menos significativos del registro, usados para hacer referencia al pin analogico del Arduino el cual se va a leer, se mantengan siempre en 0 ya que el sistema únicamente recibe señales analogicas por esta entrada, reduciendo el tiempo que se tarda en ajustar estos valores cada vez que se llama un *analogRead()*.

Seguido a esto, se configura el registro ADCSRA iniciando el bit denominado ADEN, para avisar al Arduino que se usará la entrada analógica y se inician los bits ADPS2 y ADPS0, configurando el escalador de precisión en 32 para el factor división del reloj del ADC, en vez de los 128 que usa por defecto, siendo de este cambio de donde proviene el mayor aumento en rendimiento de la optimización.

Una vez configurados estos dos registros, cada vez que se necesite hacer la lectura del voltaje generado por el sensor TMP36 únicamente es necesario iniciar el bit ADSC del registro ADCSRA el cual se encarga de enviar la señal de empezar la conversión, se espera hasta que este mismo bit se apague para señalar que la conversión terminó y se guarda el valor entero generado por el cambio de voltaje a digital por medio de los registros ADCL y ADCH, los cuales contienen los 10 bits resultado de la conversión separados entre los 8 bits menos significativos en el registro ADCL y los 2 bits más significativos restantes encontrados en la parte baja de ADCH, para extraerlos se procede a hacer un shift de 8 bits y son agregados con un OR lógico a los bits de ADCL. Este proceso es usado cada vez que es detectada una llama para determinar el nivel de peligro de esta y durante el proceso de

inicialización para calibrar el sensor de temperatura a la temperatura ambiente del lugar en el que se encuentra el sistema.

Análisis comparativo “antes y después”

La funcionalidad de lectura de temperatura dentro del módulo `flameAlarm()` fue inicialmente implementada utilizando la función `analogRead()`, una rutina provista por Arduino que encapsula el proceso completo de conversión analógica-digital. Si bien esta función proporciona una interfaz sencilla y abstrae los detalles internos del convertidor ADC, introduce una sobrecarga de procesamiento considerable debido a la configuración repetitiva de registros, el manejo interno de tiempos de estabilización y la ejecución de pasos adicionales antes y después de la conversión. Como consecuencia, cada llamada a `analogRead()` requiere aproximadamente 110 microsegundos en condiciones normales, aumentando hasta cerca de 200 microsegundos para la primera lectura tras la inicialización del sistema. Esta latencia acumulada impacta directamente la capacidad del sistema para responder de manera oportuna a un evento crítico, particularmente cuando se detecta una llama intensa que exige muestreos de temperatura más frecuentes.

En contraste, la versión optimizada prescinde por completo de la función `analogRead()` y opera directamente sobre los registros del ADC del microcontrolador AVR. Esta estrategia reduce la carga computacional al limitar el trabajo realizado en cada ciclo de muestreo únicamente a lo estrictamente necesario para iniciar y finalizar una conversión. Al configurar los registros `ADMUX` y `ADCSRA` durante la fase de inicialización, el sistema evita repetir configuraciones redundantes en cada lectura, permitiendo que el proceso completo de conversión se ejecute de forma más directa y eficiente. Además, el uso explícito del bit `ADSC` para iniciar la conversión elimina la necesidad de funciones intermedias y reduce la dependencia del tiempo de espera impuesto por la librería Arduino.

Como consecuencia de estas modificaciones, el tiempo total requerido para obtener una lectura analógica disminuye drásticamente. La conversión, que originalmente tomaba entre 110 y 200 microsegundos según el estado del sistema, se reduce a aproximadamente 32 microsegundos al trabajar directamente con el ADC. En términos de ciclos de reloj, esta optimización implica pasar de una media cercana a 1792–3328 ciclos en la versión original a aproximadamente 512 ciclos en la versión optimizada. Esto representa una mejora del orden de 3.5 a 6.5 veces en velocidad de muestreo, lo cual es especialmente relevante considerando que este módulo se ejecuta repetidamente durante la detección y evaluación de condiciones peligrosas.

Finalmente, esta disminución en la latencia de lectura no solo incrementa la eficiencia del sistema, sino que mejora de forma significativa la fiabilidad y la capacidad de reacción ante cambios bruscos en la intensidad de la llama. Un muestreo más rápido permite tomar decisiones más oportunas sobre la activación de alarmas o mecanismos de extinción, incrementando la seguridad general del sistema y su efectividad operativa. En conjunto, la optimización mediante el uso de lenguaje ensamblador constituye una mejora sustancial en términos de rendimiento, control y precisión del módulo de lectura de temperatura.

Bibliografía

Adafruit Industries. (s.f.). *TC1602A-01T: LCD module datasheet* [Ficha técnica]. Recuperado el 22 de octubre de 2025, de <https://cdn-shop.adafruit.com/datasheets/TC1602A-01T>.

Analog Devices, Inc. (s.f.). *TMP35/TMP36/TMP37: Low voltage temperature sensors* [Ficha técnica]. Recuperado el 22 de octubre de 2025, de https://dl.nm9ip6v2uc.cloudfront.net/datasheets/Sensors/Temp/TMP35_36_37.pdf

Arduino. (s/f). *Overview of the Arduino UNO Components*. Arduino.cc. Recuperado el 7 de septiembre de 2025, de <https://docs.arduino.cc/tutorials/uno-rev3/intro-to-board/>

ArduinoModules.info. (s.f.). *KY-026 flame sensor module*. ArduinoModules.info. Recuperado el 22 de octubre de 2025, de <https://arduinomodules.info/ky-026-flame-sensor-module/>

Murata Manufacturing Co., Ltd. (s.f.). *PKM13EPYH4000-A0*. Murata Manufacturing. Recuperado el 24 de octubre de 2025, de <https://pim.murata.com/en-global/pim/details/?partNum=PKM13EPYH4000-A0>

ServoDatabase.com. (s.f.). *SpringRC SM-S2309S servo specifications*. ServoDatabase.com. Recuperado el 22 de octubre de 2025, de <https://servodatabase.com/servo/springrc/sm-s2309s>

Söderby, K. (s/f). *Getting Started with Arduino*. Arduino.cc. Recuperado el 7 de septiembre de 2025, de <https://docs.arduino.cc/learn/starting-guide/getting-started-arduino/>

Soneira, E. (2016, diciembre 29). *Electrónica: Diferencias entre circuitos analógicos y circuitos digitales*. CEAC. <https://www.ceac.es/blog/electronica-diferencias-entre-circuitos-analogicos-y-circuitos-digitales>

UNO R3. (s/f). Arduino.cc. Recuperado el 7 de septiembre de 2025, de <https://docs.arduino.cc/hardware/uno-rev3/>

What are intelligent systems. (s/f). University of Nevada, Reno. Recuperado el 7 de

septiembre de 2025, de

<https://www.unr.edu/cse/undergraduates/prospective-students/what-are-intelligent-systems>